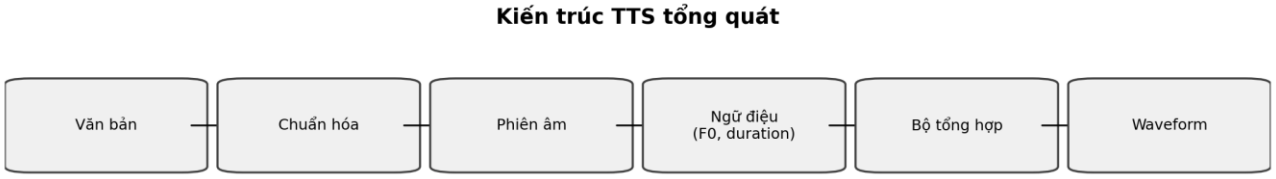
TRƯỜNG ĐẠI HỌC THỦY LỢI
KHOA CÔNG NGHỆ THÔNG TIN • BỘ MÔN TRÍ TUỆ NHÂN TẠO

LAB 4
TỔNG HỢP TIẾNG NÓI

Từ kiến trúc TTS, ghép nối và unit selection đến tổng hợp tham số HMM và TTS học sâu



Hình 1. Chuỗi xử lý tổng quát của một hệ Text-to-Speech.

1. Mục tiêu và chuẩn đầu ra của Lab

Lab 4 giúp sinh viên nối kiến thức xử lý tín hiệu, đặc trưng và mô hình xác suất với bài toán phát sinh waveform tiếng nói. Sau Lab, sinh viên phải hiểu được sự khác nhau giữa tổng hợp ghép nối, tổng hợp tham số dựa trên HMM và TTS học sâu; đồng thời tự xây dựng được một mini TTS miền hẹp và đánh giá chất lượng đầu ra.

- Mô tả được kiến trúc TTS: text analysis → phonetic analysis → prosody → synthesis backend → waveform.
- Thực hiện text normalization đơn giản, ánh xạ token sang đơn vị phát âm/đơn vị ghi âm và tạo câu nói bằng ghép nối.
- Giải thích và tính target cost, concatenation/join cost; dùng dynamic programming hoặc tìm kiếm đơn giản để chọn chuỗi unit.
- Giảm click/discontinuity tại biên bằng cross-fade/overlap-add; phân tích sự khác nhau trên waveform và spectrogram.
- Giải thích vai trò của F0, duration, energy và mô hình source–filter trong việc điều khiển giọng tổng hợp.
- Giải thích cách HMM chạy ở chế độ generation để sinh chuỗi tham số âm học theo state và thời gian.
- Chạy inference bằng một mô hình TTS học sâu pretrained và mô tả pipeline text → acoustic representation → vocoder.
- Đánh giá intelligibility, naturalness, continuity và latency; so sánh ba hướng tổng hợp.

1.1. Phân bổ 3 tiết

Thời lượng	Nội dung	Kết quả
Tiết 1	TTS pipeline, text normalization, xây inventory và baseline ghép nối.	Sinh câu nói đầu tiên từ các word-unit
Tiết 2	Unit selection, smoothing, prosody, HMM generation	Cải thiện đường nối; quan sát tham số HMM
Tiết 3	Inference TTS học sâu; so sánh, listening test	Bảng so sánh 3 phương pháp

	và báo cáo	+ file audio

1.2. Kiến thức và kỹ năng tiên quyết

- Lab 1: waveform, FFT/STFT, window, spectrogram, lọc và OLA cơ bản.
- Lab 2–3: frame, F0, MFCC, HMM và dynamic programming ở mức khái niệm.
- Python/NumPy/SciPy, đọc/ghi WAV, thao tác chuỗi, dictionary và mảng.

2. Chuẩn bị môi trường và dữ liệu

2.1. Môi trường Python

```
pip install numpy scipy matplotlib soundfile librosa
# optional: pydub (nếu muốn thao tác audio theo đoạn)
```

2.2. Dữ liệu thực hành đề xuất

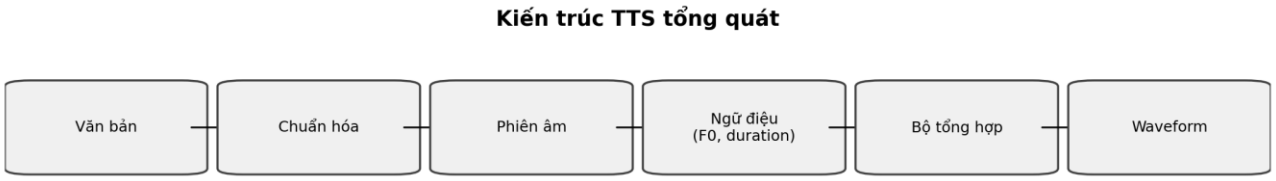
- Inventory WAV mono, Fs = 16 kHz, 16-bit PCM, mỗi file chứa một từ/đơn vị phát âm đã cắt silence tương đối sạch.
- Case study sử dụng miền hẹp "thông báo xếp hàng": mời, khách, hàng, số, không–chín, đến, quầy.
- Mỗi unit nên có 2-3 bản ghi để có thể minh họa unit selection thay vì chỉ ghép một bản cố định.
- Sinh viên tự tạo một backend HMM demo hoặc các state-parameter files, và một neural TTS pretrained/wrapper chạy offline.

```
lab4_data/
  units/
    moi_01.wav  moi_02.wav
    khach_01.wav ...
    mot_01.wav  mot_02.wav ... chin_02.wav
    den_01.wav  quay_01.wav ...
  metadata.csv
  hmm_demo/
  neural_tts_backend.py
```

3. Tóm tắt kiến thức và công thức cần dùng

3.1. Kiến trúc Text-to-Speech

Một hệ TTS điển hình nhận văn bản, chuẩn hóa về dạng "có thể đọc", chuyển sang chuỗi âm/đơn vị phát âm, gắn ngữ điệu (duration và pitch) và cuối cùng sinh waveform. Huang, Acero & Hon mô tả đúng chuỗi text analysis → phonetic analysis → prosodic analysis → speech synthesis.



Hình 2. TTS không chỉ là "đọc văn bản"; phần front-end quyết định nội dung và ngữ điệu trước khi backend sinh âm.

3.2. Text normalization và biểu diễn phát âm

Text normalization biến ký hiệu viết thành chuỗi từ nói. Trong miền hẹp, quy tắc có thể hoàn toàn xác định; ví dụ "23" có thể được đọc thành "hai ba" nếu đây là mã số, hoặc "hai mươi ba" nếu đây là số đếm. Vì vậy cùng một chuỗi ký tự có thể cần ngữ cảnh để chọn cách đọc.

$$T \xrightarrow{\text{normalize}} W = (w_1, w_2, \ldots, wK) \xrightarrow{\text{G2P/lexicon}} P = (p_1, p_2, \ldots, pL)$$

T: văn bản đầu vào; W: chuỗi token đã chuẩn hóa; P: chuỗi phone hoặc đơn vị phát âm. Trong case study miền hẹp, ta dùng word-unit nên có thể bỏ qua G2P đầy đủ.

3.3. Prosody: duration, F0 và năng lượng

Prosody quyết định nhịp, trọng âm và đường cao độ. Với tín hiệu hữu thanh, nếu chu kỳ pitch là Np mẫu ở tần số lấy mẫu Fs thì:

$$F_0 = Fs / Np$$

Ví dụ Fs = 16 kHz và Np = 100 mẫu $\Rightarrow F_0$ = 160 Hz. Giảm Np làm F0 tăng, giọng nghe cao hơn.

Có thể mô tả đơn giản một đơn vị phát âm bằng bộ tham số prosody {d, F0(t), A(t)}, trong đó d là duration, F0(t) là contour tần số cơ bản và A(t) là biên độ/energy contour. Trong TTS hiện đại, các tham số này có thể do mô hình học trực tiếp hoặc được điều khiển gián tiếp.

3.4. Mô hình source–filter cho tổng hợp tiếng nói

Tài liệu Rabiner–Schafer mô hình hóa speech bằng nguồn kích thích e[n] đi qua vocal-tract filter H(z).

$$X(z) = E(z)H(z), \quad H(z) = B(z)/A(z) = (b_0 + \Sigma\, b_k z^{-k}) / (1 - \Sigma\, a_k z^{-k})$$

Voiced: excitation gần tuần hoàn (xung theo pitch). Unvoiced: excitation gần nhiễu. Các hệ số lọc thay đổi theo thời gian và quyết định spectral envelope/formant.

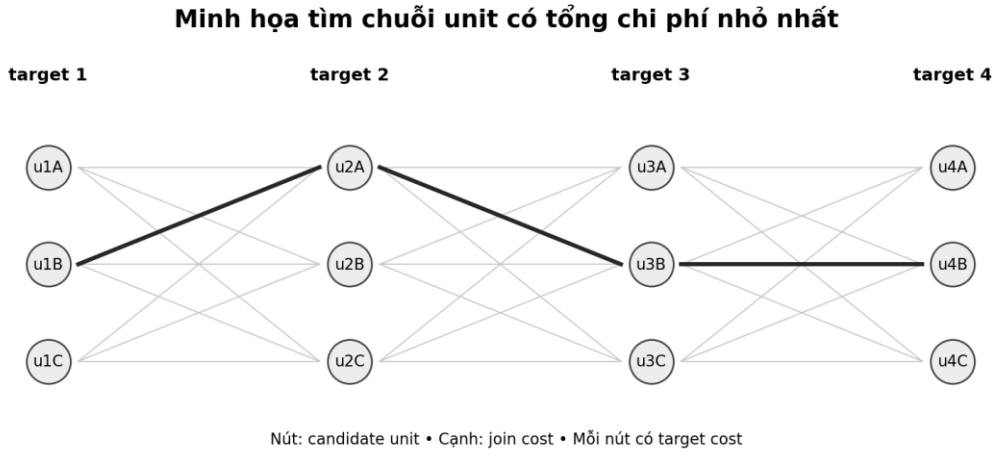
Trong nhiều synthesizer tham số, backend thực chất dự đoán/điều khiển các tham số nguồn và lọc rồi dùng vocoder để phát sinh waveform.

3.5. Concatenative synthesis và unit selection

Concatenative synthesis phát lại các đoạn tiếng nói tự nhiên rồi ghép chúng thành câu. Đơn vị có thể là word, syllable, phone, diphone… Dữ liệu lớn giúp tăng số context và naturalness nhưng làm bài toán chọn unit phức tạp hơn.

$$\theta^* = \arg\min_{\{\theta_1 \ldots \theta J\}} \left[\Sigma_j\, C_target(\theta_j, t_j) + \lambda\, \Sigma_j\, C_join(\theta_j, \theta_{j+1}) \right]$$

t_j: unit mong muốn tại vị trí j; θ_j: candidate lấy từ inventory. Target cost đo mức phù hợp với mục tiêu; join cost đo độ "êm" khi nối hai candidate; λ điều chỉnh cân bằng.



Hình 3. Unit selection có thể xem như bài toán tìm đường chi phí thấp nhất qua lưới candidate.

Một join cost phổ biến là normalized mean-square mismatch của feature tại hai đầu unit:

$$C^{c}_{k}(\theta_j,\theta_{j+1}) = [\ f_k^{\{\theta_j\}}(m) - f_k^{\{\theta_{j+1}\}}(1)\ ]^2 / \sigma_k^2$$

f_k có thể là log-energy, F0 hoặc feature phổ; σ_k^2 chuẩn hóa theo độ biến thiên tự nhiên của feature k. Tổng join cost thường là tổng có trọng số trên nhiều feature.

3.6. Boundary smoothing: cross-fade, OLA và PSOLA

Ghép thẳng hai waveform dễ sinh click do lệch biên độ/pha. Cross-fade trong vùng N mẫu dùng một ramp α[n] từ 0 đến 1:

$$y[n] = (1 - \alpha[n])x_1[n] + \alpha[n]x_2[n], \quad 0 \le n < N$$

Cross-fade 10–30 ms thường đủ để giảm discontinuity ở biên word-unit miền hẹp; quá dài có thể làm mờ phụ âm chuyển tiếp.

Với overlap-add, các frame đã nhân window được dịch và cộng:

$$y[n] = \Sigma_r\ x_r[n] = x[n]\ \Sigma_r\ w[rR - n]$$

Nếu tổng các cửa sổ chồng lấn là hằng số, tín hiệu có thể được tái tạo không méo theo biên độ. Đây là nguyên lý nền cho OLA/WSOLA/PSOLA.

Trong TD-PSOLA, các đoạn quanh pitch epoch được nhân window và đặt lại ở vị trí tổng hợp mới. Việc thay khoảng cách giữa các epoch thay F0 nhưng cố gắng giữ spectral envelope tương đối ổn định.

$$s_i[n] = s[n]\ w[n - iT_0]$$

T_0: pitch period gốc. Khi các segment $s_i[n]$ được dịch rồi overlap-add với khoảng T mới, pitch period đầu ra xấp xỉ T.

3.7. Tổng hợp tham số dựa trên HMM

Trong hướng HMM-based synthesis, HMM không dùng để nhận dạng mà chạy ở chế độ generation: chuỗi phone/context chọn một chuỗi state; mỗi state mang phân bố của các tham số âm học. Huang et al. mô tả HMM phát formant và bandwidth mỗi 10 ms; khi bổ sung delta feature, nghiệm maximum-likelihood trở nên trơn hơn giữa state.

$$\lambda = (A, B, \pi), \quad \hat{O}_t = [c_t, \Delta c_t] \quad \text{và} \quad \hat{O}^* = \arg\max_O P(O \mid \lambda, \text{state sequence})$$

c_t có thể là vector phổ/formant/MFCC-like; Δc_t ràng buộc độ biến thiên. Trong hệ thực, duration model và F0 model thường được xử lý riêng hoặc phối hợp.

3.8. Đánh giá chất lượng TTS

$$MOS = (1/K)\ \Sigma_i\ r_i, \quad r_i \in \{1,2,3,4,5\}$$

MOS là trung bình điểm chủ quan của K lượt nghe. Nên tách câu hỏi naturalness khỏi intelligibility nếu có thể.

$$\text{Intelligibility (\%)} = 100 \times N_correct / N_total$$

Có thể dùng số từ nghe đúng hoặc số câu nghe đúng. Với Lab nhỏ, listening test mù A/B hoặc MOS mini 5–10 người là phù hợp.

- Naturalness: giống giọng người, ít robotic/buzzy.
- Continuity: ít click, ít jump phổ/pitch tại biên.
- Intelligibility: nghe đúng nội dung.
- Latency và tài nguyên: thời gian từ text tới lúc có audio, kích thước inventory/model.

4. Yêu cầu thực hành bắt buộc

4.1. A - Text normalization và inventory

- Xây hàm normalize_text() cho miền hẹp: lowercase, bỏ dấu câu không cần thiết, tách token và đọc chuỗi số theo quy tắc của bài.
- Tạo inventory ít nhất 15 từ/đơn vị với 2 candidate/unit nếu có thể.
- Chuẩn hóa Fs, mono và RMS của các unit; lưu metadata gồm label, speaker, duration, RMS và file path.
- Vẽ histogram duration và RMS để kiểm tra bất thường dữ liệu.

4.2. B - Baseline concatenation

- Ghép trực tiếp một candidate cố định cho từng token; lưu raw_concat.wav.
- Đánh dấu vị trí biên nối và vẽ waveform + spectrogram.
- Nghe và ghi nhận click, chênh energy, pitch jump hoặc thay đổi spectral envelope.

4.3. C - Unit selection và smoothing

- Tính target cost đơn giản từ duration/RMS mong muốn và join cost từ 20–30 ms ở cuối/đầu hai unit.
- Nếu mỗi vị trí có >1 candidate, tìm chuỗi cost nhỏ nhất bằng dynamic programming hoặc exhaustive search nếu câu rất ngắn.
- Thử cross-fade 0, 5, 15, 30 ms; chọn giá trị hợp lý dựa trên nghe và spectral discontinuity.
- Lưu best_concat.wav và bảng total cost.

4.4. D - Prosody và source–filter

- Đo F0 của ít nhất một unit hữu thanh; kiểm chứng F0≈Fs/Np trên một đoạn gần tuần hoàn.
- Thử thay speed/duration bằng WSOLA/time-stretch (nếu thư viện hỗ trợ) và pitch shift ±2 semitone; so sánh spectrogram.
- Giải thích tại sao thay duration và pitch không nên thay spectral envelope một cách tùy ý.

4.5. E - HMM-based synthesis: bài quan sát có hướng dẫn

- Dùng backend/model đã tạo ở trên để lấy state sequence và các trajectory F0/duration/spectral parameter của một câu.
- Vẽ ít nhất F0 contour và một spectral/formant parameter theo frame.
- Thay một tham số (ví dụ scale duration 0.8/1.2 hoặc F0 0.9/1.1) rồi nghe đầu ra nếu backend hỗ trợ resynthesis.
- Giải thích sự khác nhau giữa "chọn waveform tự nhiên" và "sinh trajectory tham số từ phân bố HMM".

4.6. F - TTS học sâu bằng mô hình pretrained

- Chạy cùng 3 câu test qua neural TTS pretrained offline.
- Lưu waveform và mel-spectrogram (nếu backend trả về).
- Đo latency từ lúc nhận text đến khi sinh xong audio trên cùng máy.
- Không tải model ngẫu nhiên từ Internet trong giờ Lab.

4.7. G - Đánh giá và so sánh

Phương pháp	Intelligibility	Naturalness	Continuity	Latency	Bộ nhớ/dữ liệu
Concatenation					

HMM/parametric					
Neural TTS					

5. Case study - Mini TTS “Thông báo xếp hàng”

Mục tiêu: từ câu văn bản “Mời khách hàng số 23 đến quầy 5.”, tạo audio bằng ghép nối word-unit, cải thiện biên nối, rồi so sánh với backend HMM và neural TTS đã tạo ở trên.



Hình 6. Luồng case study miền hẹp.

5.1. Chuẩn bị inventory

```
units = {
  "mời": ["moi_01.wav", "moi_02.wav"],
  "khách": ["khach_01.wav", "khach_02.wav"],
  "hàng": ["hang_01.wav", "hang_02.wav"],
  "số": ["so_01.wav", "so_02.wav"],
  "không": [...], "một": [...], ..., "chín": [...],
  "đến": [...], "quầy": [...]
}
```

- Ghi cùng microphone, cùng khoảng cách, cùng Fs=16 kHz.
- Mỗi word có 100–200 ms context silence khi thu, sau đó trim về mức hợp lý.
- Không normalize quá mạnh từng unit; giữ khác biệt tự nhiên nhỏ nhưng tránh clipping.

5.2. Chuẩn hóa câu

```
DIGIT = {"0":"không","1":"một","2":"hai","3":"ba","4":"bốn",
        "5":"năm","6":"sáu","7":"bảy","8":"tám","9":"chín"}

def normalize_queue_text(text):
    # Miền hẹp: mọi chuỗi số được đọc từng chữ số.
    text = text.lower().replace(".", "").replace(",", "")
    out = []
    for tok in text.split():
        if tok.isdigit():
            out.extend(DIGIT[c] for c in tok)
        else:
            out.append(tok)
    return out

# → ["mời","khách","hàng","số","hai","ba","đến","quầy","năm"]
```

5.3. Ghép thẳng và cross-fade

```
import numpy as np, soundfile as sf
```

```
def load_unit(path, sr=16000):
    x, fs = sf.read(path, dtype="float32")
    if x.ndim > 1: x = x.mean(axis=1)
    assert fs == sr
    return x

def rms(x):
    return np.sqrt(np.mean(x*x) + 1e-12)

def match_rms(x, target=0.08):
    y = x * (target / rms(x))
    return np.clip(y, -0.99, 0.99)

def crossfade(a, b, n):
    n = min(n, len(a), len(b))
    if n == 0: return np.concatenate([a,b])
    alpha = np.linspace(0.0, 1.0, n, endpoint=True)
    mid = (1-alpha)*a[-n:] + alpha*b[:n]
    return np.concatenate([a[:-n], mid, b[n:]])

def concat_units(paths, sr=16000, xfade_ms=15):
    xs = [match_rms(load_unit(p, sr)) for p in paths]
    y = xs[0]
    n = int(sr*xfade_ms/1000)
    for x in xs[1:]:
        y = crossfade(y, x, n)
    return y

# sf.write("best_concat.wav", y, 16000)
```

Sinh viên phải chạy hai bản: xfade_ms=0 và xfade_ms=15 (sau đó thí nghiệm 5/30 ms), nghe mù và so sánh waveform/spectrogram tại ít nhất hai biên nối.

5.4. Join-cost đơn giản để chọn candidate

$$D_{spec} = (1/K)\ \Sigma_k\ [\ L_{left}[k] - L_{right}[k]\ ]^2$$

L_left, L_right là log-magnitude spectrum của 20–30 ms ở hai phía biên. Có thể cộng thêm |RMS_left−RMS_right| và |F0_left−F0_right| sau khi chuẩn hóa.

```
def join_cost(a, b, sr=16000, ms=25, nfft=512):
    n = int(sr*ms/1000)
    wa = np.hanning(n); wb = np.hanning(n)
    A = np.fft.rfft(a[-n:]*wa, n=nfft)
    B = np.fft.rfft(b[:n]*wb, n=nfft)
    La = 20*np.log10(np.maximum(np.abs(A), 1e-8))
    Lb = 20*np.log10(np.maximum(np.abs(B), 1e-8))
    return float(np.mean((La-Lb)**2))
```

5.5. Backend HMM - cách làm trong Lab

```
# API minh họa; file backend do đã tạo
from hmm_tts_backend import synthesize_hmm

y_hmm, sr, tracks = synthesize_hmm("Mời khách hàng số 23 đến quầy 5")
# tracks có thể gồm: F0, duration, spectral/formant trajectory
# Sinh viên vẽ tracks["f0"] và ít nhất một trajectory khác.
```

Sinh viên hoàn thành mục này bằng cách quan sát state/parameter trajectory và giải thích quá trình generation; không bắt buộc tự tái tạo waveform từ MFCC.

5.6. Backend neural TTS - cách làm trong Lab

```
# API minh họa; model/wrapper được đóng gói offline trước giờ Lab
from neural_tts_backend import synthesize

y_nn, sr, extra = synthesize("Mời khách hàng số 23 đến quầy 5")
# extra có thể chứa mel-spectrogram hoặc thời gian inference
sf.write("neural_tts.wav", y_nn, sr)
```

5.7. Kết quả case study tối thiểu

- raw_concat.wav và best_concat.wav.
- Waveform/spectrogram trước và sau cross-fade, có đánh dấu biên unit.
- Bảng join cost của ít nhất 2 cách chọn candidate.
- Một biểu đồ F0/parameter trajectory từ backend HMM.
- neural_tts.wav và latency.
- Bảng đánh giá A/B hoặc MOS mini cho ít nhất 3 câu.

6. Thí nghiệm và phân tích bắt buộc

TN	Biến thay đổi	Giá trị gợi ý	Cần trả lời
1	Cross-fade	0, 5, 15, 30 ms	Mức nào giảm click mà ít làm mờ phụ âm nhất?
2	Candidate/unit	1 so với 2–3 candidate	Unit selection có giảm join cost và tăng naturalness không?
3	Duration/speed	0.8×, 1.0×, 1.2×	Tốc độ ảnh hưởng intelligibility/naturalness thế nào?
4	Pitch	−2, 0, +2 semitone	Pitch shift có tạo artifact/buzziness không?
5	Backend	concat / HMM / neural	Phương pháp nào tốt nhất theo từng tiêu chí?

Quan hệ semitone–tỷ số tần số:

$$r = 2^{(\Delta s/12)}, \qquad F_{0,new} = r \cdot F_{0,old}$$

Δs là số semitone. Ví dụ +12 semitone $\Rightarrow r=2$, tức tăng một octave.

7. Câu hỏi báo cáo

- 1. Vì sao một câu ghép từ các đoạn "tự nhiên" vẫn có thể nghe kém tự nhiên?
- 2. Phân biệt target cost và join cost. Nếu chỉ tối ưu join cost thì có thể xảy ra vấn đề gì?
- 3. Vì sao cross-fade quá dài có thể làm giảm độ rõ của phụ âm?
- 4. Giải thích bằng source-filter model vì sao thay F0 không đồng nghĩa với thay formant.
- 5. PSOLA thay pitch bằng cách nào? Tại sao voiced fricative có thể bị buzziness khi kéo dài quá mức?
- 6. Trong HMM-based synthesis, vì sao thêm delta feature làm trajectory trơn hơn?
- 7. Tại sao whole waveform concatenation cần inventory lớn để đạt naturalness cao?
- 8. Nêu ít nhất hai khác biệt cơ bản giữa concatenative TTS và neural TTS.
- 9. Vì sao không nên so sánh neural TTS bằng một model tùy ý tải từ Internet trong buổi Lab?
- 10. Nếu một hệ thống có MOS cao nhưng intelligibility thấp thì có thể dùng trong ứng dụng nào? Giải thích.

8. Sản phẩm nộp

- Lab04.ipynb hoặc Lab04.py có thể chạy lại từ đầu với cấu trúc thư mục rõ ràng.

- Folder audio_output gồm raw_concat.wav, best_concat.wav, hmm_tts.wav (nếu backend cho phép) và neural_tts.wav.
- Ít nhất 5 hình: inventory stats, spectrogram biên ghép, unit-selection/join cost, HMM track, so sánh output.
- Báo cáo 5–8 trang: mô tả phương pháp, công thức đã dùng, thiết kế thí nghiệm, kết quả, nghe chủ quan và kết luận.
- File ratings.csv nếu nhóm tổ chức listening test.

9. Rubric chấm điểm (10 điểm)

Hạng mục	Điểm	Tiêu chí
Chuẩn bị dữ liệu & normalization	1.0	Inventory đúng format, audio sạch, quy tắc normalization rõ.
Concatenative baseline	1.5	Sinh được câu đúng nội dung, đánh dấu/giải thích biên nối.
Unit selection & smoothing	2.0	Có cost hợp lý, cross-fade/OLA đúng và phân tích trước/sau.
Prosody/source–filter	1.0	Thí nghiệm pitch/duration và giải thích đúng.
HMM-based synthesis	1.5	Hiểu state/trajectory, vẽ và phân tích tham số.
Neural TTS inference	1.0	Chạy đúng model offline, lưu output/latency.
Đánh giá & báo cáo	2.0	So sánh có dữ liệu, biểu đồ/audio minh họa và kết luận có căn cứ.

10. Các lỗi thường gặp và cách kiểm tra

Lỗi	Nguyên nhân	Cách kiểm tra/sửa
Click ở biên	DC/RMS/phase khác nhau	Trim zero crossing tương đối; match RMS; cross-fade.
Âm bị nhỏ/dồn	Cross-fade cộng/trừ sai	Kiểm tra ramp α và độ dài overlap.
Câu sai nội dung	Normalization/token mapping sai	In token sequence trước synthesis.
Pitch shift làm "chipmunk"	Đổi cả pitch và spectral envelope	Dùng PSOLA/vocoder tách source–filter nếu có.
HMM trajectory giật	Không dùng dynamic feature hoặc scale sai	Kiểm tra Δ-feature và frame rate.
Neural TTS chạy khác máy khác kết quả	Khác model/version/device	Đóng gói model và version trước Lab.
So sánh không công bằng	Câu khác nhau/âm lượng khác	Dùng cùng câu, normalize loudness và test mù.

11. Tài liệu tham khảo và phạm vi nguồn

- X. Huang, A. Acero, H.-W. Hon, Spoken Language Processing, Prentice Hall, 2001 - Ch. 14 Text and Phonetic Analysis, Ch. 15 Prosody, Ch. 16 Speech Synthesis.

- L. R. Rabiner, R. W. Schafer, Theory and Applications of Digital Speech Processing, Prentice Hall, 2010 - Ch. 13 Text-to-Speech Synthesis Methods; các phần source–filter, unit selection và TD-PSOLA.